

Fine-Tuning in Machine Learning

Introduction

Fine-tuning is a technique where a **pre-trained model** (trained on a large dataset) is further trained on a smaller, **task-specific dataset**. This allows the model to adapt to the specific task while leveraging the knowledge it has already learned from the larger dataset.

Why Use Fine-Tuning?

- **Saves Time and Resources:** Training a model from scratch on a large dataset requires significant computational resources and time. Fine-tuning avoids this by starting with a pre-trained model.
- **Improves Performance:** Pre-trained models have already learned useful features from large datasets. Fine-tuning allows the model to specialize these features for a specific task.
- **Works Well with Small Datasets:** Fine-tuning is especially useful when you have a small dataset, as the pre-trained model provides a strong starting point.

Mathematical Framework

Pre-Trained Model

Let the pre-trained model have h layers with weights $\mathbf{W}^* = \{\mathbf{W}_1^*, \mathbf{W}_2^*, \dots, \mathbf{W}_h^*\}$. These weights are optimized for a general task (e.g., language modeling for BERT or image classification for ResNet).

Task-Specific Model

For the new task, we add a **task-specific head** (e.g., a fully connected layer for classification) with weights $\mathbf{W}_{\text{head}}$. The task-specific head is randomly initialized.

Fine-Tuning Objective

The goal is to minimize the task-specific loss $\mathcal{L}(\mathbf{W}, \mathbf{W}_{\text{head}})$, where:

- $\mathbf{W} = \{\mathbf{W}_1, \mathbf{W}_2, \dots, \mathbf{W}_h\}$ are the weights of the pre-trained model.
- $\mathbf{W}_{\text{head}}$ are the weights of the task-specific head.

Freezing Layers

What Does Freezing Mean?

Freezing a layer means **fixing its weights** during fine-tuning. The gradients are not computed for frozen layers, and their weights are not updated.

Why Freeze Layers?

- **Retain General Features:** Early layers of a pre-trained model often learn general features (e.g., edge detectors in CNNs or word embeddings in NLP). Freezing these layers preserves these features.
- **Prevent Overfitting:** Freezing layers reduces the number of trainable parameters, which is especially useful for small datasets.
- **Save Computational Resources:** Fewer trainable parameters mean faster training and lower memory usage.

Which Layers to Freeze?

- **Early Layers:** Early layers typically learn low-level features (e.g., edges, textures, or word embeddings). These features are often transferable across tasks, so freezing them is common.
- **Later Layers:** Later layers learn task-specific features. These are often fine-tuned to adapt to the new task.

Practical Strategies for Freezing Layers

1. Freeze Early Layers

Freeze the first k layers (e.g., $k = 8$ out of $h = 12$ for BERT-base). Fine-tune the remaining $h - k$ layers and the task-specific head.

2. Gradually Unfreeze Layers

Start by freezing all layers and fine-tuning only the task-specific head. Gradually unfreeze layers from the top (later layers) to the bottom (earlier layers) as training progresses.

3. Layer-Wise Learning Rates

Use different learning rates for different layers:

- Lower learning rates for early layers (to preserve general features).
- Higher learning rates for later layers and the task-specific head (to adapt to the new task).

4. Experiment and Validate

The optimal number of layers to freeze depends on:

- The size of your dataset.
- The similarity between the pre-training task and the new task.
- Use cross-validation to determine the best strategy.

Mathematical Details of Freezing

1. Frozen Layers

For frozen layers, the weights $\mathbf{W}_i$ are fixed:

$$\mathbf{W}_i \leftarrow \mathbf{W}_i \quad \text{for } i = 1, 2, \dots, k$$

No gradients are computed for these layers:

$$\nabla_{\mathbf{W}_i} \mathcal{L}(\mathbf{W}, \mathbf{W}_{\text{head}}) = 0 \quad \text{for } i = 1, 2, \dots, k$$

2. Trainable Layers

For trainable layers, the weights $\mathbf{W}_i$ are updated:

$$\mathbf{W}_i \leftarrow \mathbf{W}_i - \eta \nabla_{\mathbf{W}_i} \mathcal{L}(\mathbf{W}, \mathbf{W}_{\text{head}}) \quad \text{for } i = k+1, k+2, \dots, h$$

The task-specific head $\mathbf{W}_{\text{head}}$ is also updated:

$$\mathbf{W}_{\text{head}} \leftarrow \mathbf{W}_{\text{head}} - \eta \nabla_{\mathbf{W}_{\text{head}}} \mathcal{L}(\mathbf{W}, \mathbf{W}_{\text{head}})$$

Example: Fine-Tuning BERT for Spam Detection

1. Pre-trained BERT Model

BERT has $h = 12$ layers (for BERT-base) with pre-trained weights $\mathbf{W}^* = \{\mathbf{W}_1^*, \mathbf{W}_2^*, \dots, \mathbf{W}_{12}^*\}$.

2. Freeze Early Layers

Freeze the first $k = 8$ layers:

$$\mathbf{W}_i \leftarrow \mathbf{W}_i \quad \text{for } i = 1, 2, \dots, 8$$

Fine-tune the remaining 4 layers and the task-specific head.

3. Task-Specific Head

Add a fully connected layer with a sigmoid activation:

$$\mathbf{W}_{\text{head}} \in \mathbb{R}^{768 \times 1}$$

$\mathbf{W}_{\text{head}}$ is randomly initialized.

4. Fine-Tuning Process

Compute the forward pass:

$$\mathbf{h} = \text{BERT}(\mathbf{x}; \mathbf{W})$$

$$\mathbf{z} = \mathbf{W}_{\text{head}}^\top \mathbf{h}$$

$$\hat{y} = \sigma(\mathbf{z})$$

Compute the loss:

$$\mathcal{L}(\mathbf{W}, \mathbf{W}_{\text{head}}) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Update the trainable weights:

$$\mathbf{W}_i \leftarrow \mathbf{W}_i - \eta \nabla_{\mathbf{W}_i} \mathcal{L}(\mathbf{W}, \mathbf{W}_{\text{head}}) \quad \text{for } i = 9, 10, 11, 12$$

$$\mathbf{W}_{\text{head}} \leftarrow \mathbf{W}_{\text{head}} - \eta \nabla_{\mathbf{W}_{\text{head}}} \mathcal{L}(\mathbf{W}, \mathbf{W}_{\text{head}})$$

Key Takeaways

- Fine-tuning involves adapting a pre-trained model to a new task by updating some or all of its weights.
- Freezing layers is a common strategy to retain general features, prevent overfitting, and save resources.
- The optimal number of layers to freeze depends on the dataset size, task similarity, and computational resources.
- Experimentation and validation are key to determining the best fine-tuning strategy.